

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ЛАБОРАТОРНАЯ РАБОТА №1
по дисциплине «Вычислительная математика»
Тема: методы решения нелинейных уравнений

Студент гр. 4341

Четвертная В.Л.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Целью лабораторной работы являются изучение методов бисекции, хорд, простых итераций и метода Ньютона для поиска корня уравнения с заданной точностью, написание программы, реализующей данный метод, и исследование скорости сходимости и обусловленности метода (чувствительности к ошибкам в исходных данных).

Задание.

Требуется найти корень уравнения $f(x)=0$ указанными методами с заданной точностью Eps , исследовать скорость сходимости и обусловленность метода (чувствительность к ошибкам в исходных данных).

Порядок выполнения работы должен быть следующим:

1) Графически или аналитически отделить корень уравнения $f(x)=0$ (т.е. найти отрезки $[Left, Right]$, на которых функция $f(x)$ удовлетворяет условиям теоремы Коши).

2) Составить подпрограмму вычисления функции $f(x)$.

3) Для метода бисекции:

1. Составить головную программу, содержащую обращение к подпрограмме $f(x)$, *BISECT*, *Round* и индикацию результатов.
2. Провести вычисления по программе. Построить график зависимости числа итераций от Eps .
3. Исследовать чувствительность метода к ошибкам в исходных данных. Ошибки в исходных данных моделировать с использованием программы *Round*, округляющей значения функции с заданной точностью $Delta$.

3) Для метода хорд:

1. Составить головную программу, содержащую обращение к подпрограмме $f(x)$, *HORDA*, *Round* и индикацию результатов.
2. Провести вычисления по программе. Теоретически и экспериментально исследовать скорость сходимости и обусловленность метода.

3. Исследовать чувствительность метода к ошибкам в исходных данных. Ошибки в исходных данных моделировать с использованием программы *Round*, округляющей значения функции с заданной точностью *Delta*.

4) Для метода простых итераций:

1. Преобразовать уравнение $f(x)=0$ к виду $x=\varphi(x)$ так, чтобы в некоторой окрестности $[Left, Right]$ корня производная $\varphi'(x)$ удовлетворяла условию $\varphi'(x) \leq q < 1$. При этом следует иметь в виду, что чем меньше величина q , тем быстрее последовательные приближения сходятся к корню.
2. Выбрать начальное приближение, лежащее на $[Left, Right]$.
3. Составить подпрограмму для вычисления значений $\varphi(x)$, $\varphi'(x)$, предусмотрев округление вычисленных значений с точностью *Delta*.
4. Составить главную программу, вычисляющую корень уравнения и содержащую обращение к программам $\varphi(x)$, $\varphi'(x)$, и *ITER* и индикацию результатов.
5. Провести вычисления по программе. Исследовать скорость сходимости и обусловленность метода.

5) Для метода Ньютона:

1. Составить подпрограммы функции вычисления $f(x)$, $f'(x)$, предусмотрев округление их значений с заданной точностью *Delta*.
2. Составить главную программу, содержащую обращение к подпрограммам $f(x)$, $f'(x)$, *NEWTON*, *Round* и индикацию результатов.
3. Выбрать начальное приближение корня x_0 из $[Left, Right]$ так, чтобы $f(x)$, $f'(x) > 0$.
4. Провести вычисления по программе. Теоретически и экспериментально исследовать скорость сходимости и обусловленность метода.

Вариант 23: $f(x) = \sinh(x) - x + 1$

Отделение корней уравнения.

Рассмотрим функцию $f(x) = \sinh(x) - x + 1$ и её поведение. Она определена и непрерывна на $(-\infty; +\infty)$. Её производная равна:

$$f'(x) = \cosh(x) - 1$$

Производная равна нулю в точке $x = 0$, на остальном участке $(-\infty; 0) \cup (0; +\infty)$ положительна. Следовательно 0 является точкой перегиба функции, на всей области определения функция монотонно возрастает.

При $x \rightarrow -\infty: f(x) \rightarrow -\infty$, при $x \rightarrow +\infty: f(x) \rightarrow +\infty$.

Функция непрерывна на $(-\infty; +\infty)$ и меняет знак с $-\infty$ на $+\infty$. По теореме Больцано-Коши корень существует. Так как функция строго возрастает, корень единственный.

Локализуем корень:

$$x = -2: f(-2) = \sinh(-2) + 2 - 1 \approx -2.627 < 0$$

$$x = -1: f(-1) = \sinh(-1) + 1 - 1 \approx 0.825 > 0$$

Аналитические расчёты подтверждаются графически (см. рис.1).

Для метода бисекции возьмём отрезок $[-2; -1]$.

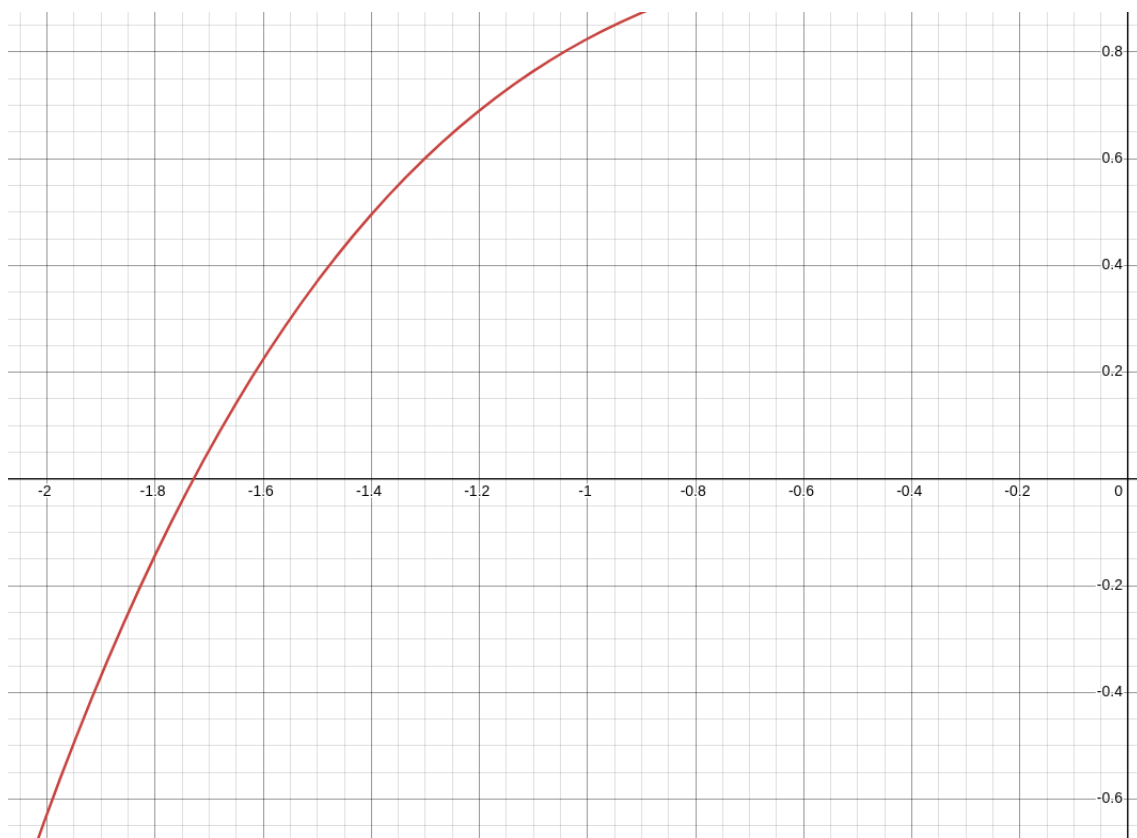


Рис. 1 - График исследуемой функции

Преобразование уравнения к итерационному виду $x = \varphi(x)$

Уравнение $f(x)=0$ можно преобразовать к виду $x=x-\lambda*f(x)$. Тогда $\varphi(x)=x-\lambda*f(x)$. Производная $\varphi'(x)=1-\lambda*f'(x)$. Подберем параметр λ так, чтобы $|\varphi'(x)|$ был минимален на отрезке.

На отрезке $[-2, -1]$: $f'(x)$ меняется от $\cosh(-2)-1=\cosh(2)-1\approx 3.762-1=2.762$ до $\cosh(-1)-1=\cosh(1)-1\approx 1.543-1=0.543$.

Оценка $f'(x)$ посередине, например в точке -1.5 : $\cosh(1.5)-1\approx 2.352-1=1.352$.

Необходимо, чтобы $\varphi'(x)=1-\lambda*f'(x)$ было как можно ближе к нулю на всем отрезке. Идеальное значение $\lambda=1/f'(x)$, но оно зависит от x . Возьмем постоянное λ , приблизительно равное $1/f'(x)$ в какой-то точке отрезка.

Пусть $\lambda=0.5$, тогда $\varphi(x)=1.5*x-0.5*\sinh(x)-0.5$, удовлетворяет условию $|\varphi'(x)| \leq 0.73 < 1$ на отрезке $[-2, -1]$.

Разработка программ.

Программы написана на языке программирования C++.

Для вычисления значения $f(x)$ написана функция *double comp_func(double x)*:

```
double comp_func(double x){  
    return sinh(x)-x+1;  
}
```

Метод бисекции реализован в функции *double Bisect(double left, double right, double eps, int &n)*

Аргументы:

- *double left, double right* – границы исследуемого отрезка;
- *double eps* – точность вычисления корня;
- *int &n* – ссылка на переменную для сохранения кол-ва итераций.

Метод хорд реализован в функции *double Horda(double left, double right, double eps, int& n)*

Аргументы:

- *double left, double right* – границы исследуемого отрезка;
- *double eps* – точность вычисления корня;
- *int& n* – ссылка на переменную для сохранения кол-ва итераций.

Метод простых итераций реализован в функции *double Iter(double x, double eps, int& n)*

Аргументы:

- *double x* – стартовое приближение корня;
- *double eps* – точность вычисления;
- *int& n* – ссылка на переменную для сохранения кол-ва итераций.

Для вычисления значения $f'(x)$ написана функция *double comp_deriv(double x)*:

```
double comp_deriv(double x){  
    return cosh(x)-1;  
}
```

Метод Ньютона реализован в функции *double Newton(double x, double eps, int& n)*

Аргументы:

- *double x* – стартовое приближение корня;
- *double eps* – точность вычисления;
- *int& n* – ссылка на переменную для сохранения кол-ва итераций.

Вычисление округления значений с заданной точностью происходит в функции *double Round(double x, double delta)*

Аргументы:

- *double x* – значения для округления;
- *double delta* – точность округления.

Код программ приведён в Приложении А.

Тестирование.

Точность вычислений	Результат
0.1	-1.6
0.01	-1.73
0.001	-1.729
0.0001	-1.7291
0.00001	-1.72911
0.000001	-1.729116

Исследование зависимости числа итераций от точности вычислений для метода бисекции.

На графике (см. рис.2) приведено сравнение полученной при работе программы зависимости количества итераций от заданной точности

вычислений с рассчитанными теоретически значениями: $N \approx \log_2 \frac{b-a}{Eps}$. Видно, что практические значения с учётом погрешностей совпадают с теоретическими, следовательно алгоритм реализован корректно.

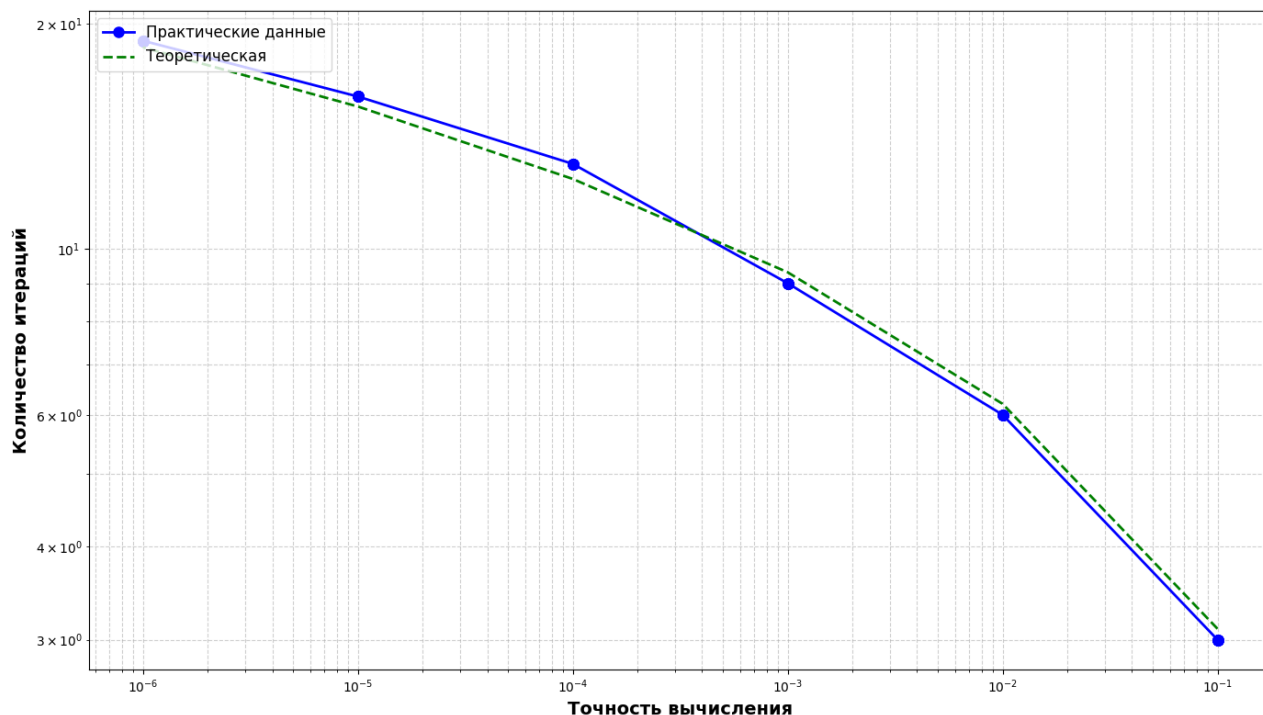


Рис. 2 - Зависимость количества итераций от точности вычислений

Исследование скорости сходимости для метода хорд.

На графике (см. рис.3) приведено исследование скорости сходимости. Скорость сходимости для метода хорд линейная и вычисляется по формуле $C \approx \frac{|x - x_{n+1}|}{|x - x_n|}$, где x_{n+1} — значение корня на текущей итерации, x_n — значение корня на предыдущей итерации, x — эталонное значение корня. В исследовании рассматривается скорость сходимости для вычисления корня с точностью 10^{-6} .

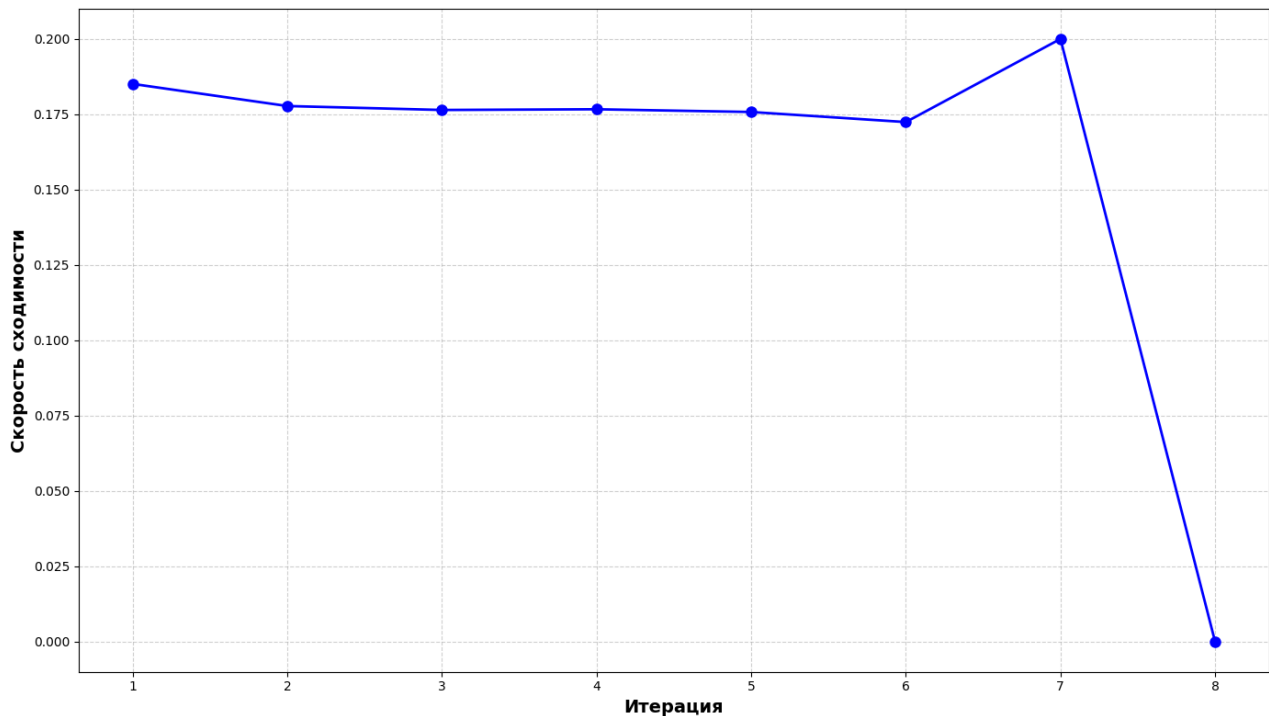


Рис. 3 - Исследование скорости сходимости

Исследование скоростей сходимости методов простых итераций и Ньютона.

На графике (см. рис.4) приведено исследование скорости сходимости. Скорость сходимости для методов простых итераций и Ньютона квадратическая и вычисляется по формуле $C \approx \frac{|x - x_{n+1}|}{|x - x_n|^2}$, где x_{n+1} — значение корня на текущей итерации, x_n — значение корня на предыдущей итерации, x — эталонное значение корня. В исследовании рассматривается скорость сходимости для вычисления корня с точностью 10^{-6} .

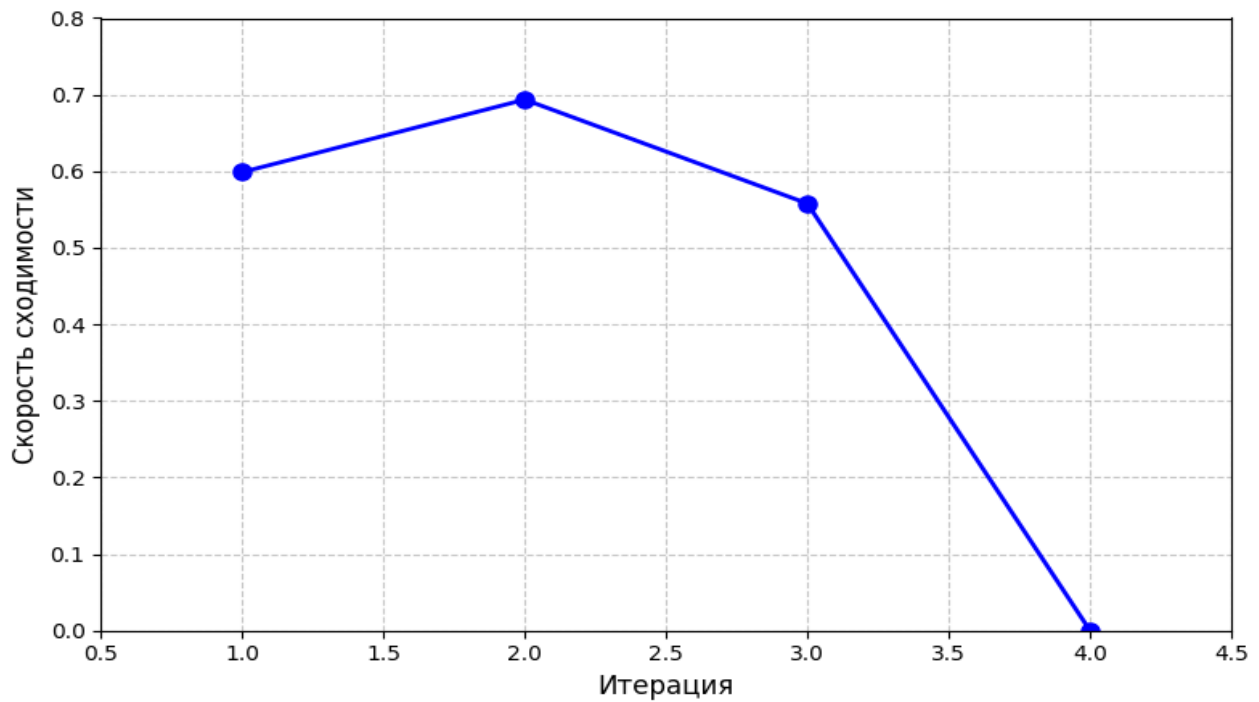


Рис. 4 - Исследование скорости сходимости

Исследование обусловленности методов.

На графике (см. рис.5) приведена зависимость результата вычислений от точности исходных данных, получены несколько кривых, так как для каждой точности исходных данных вычислены результаты с различной точностью.

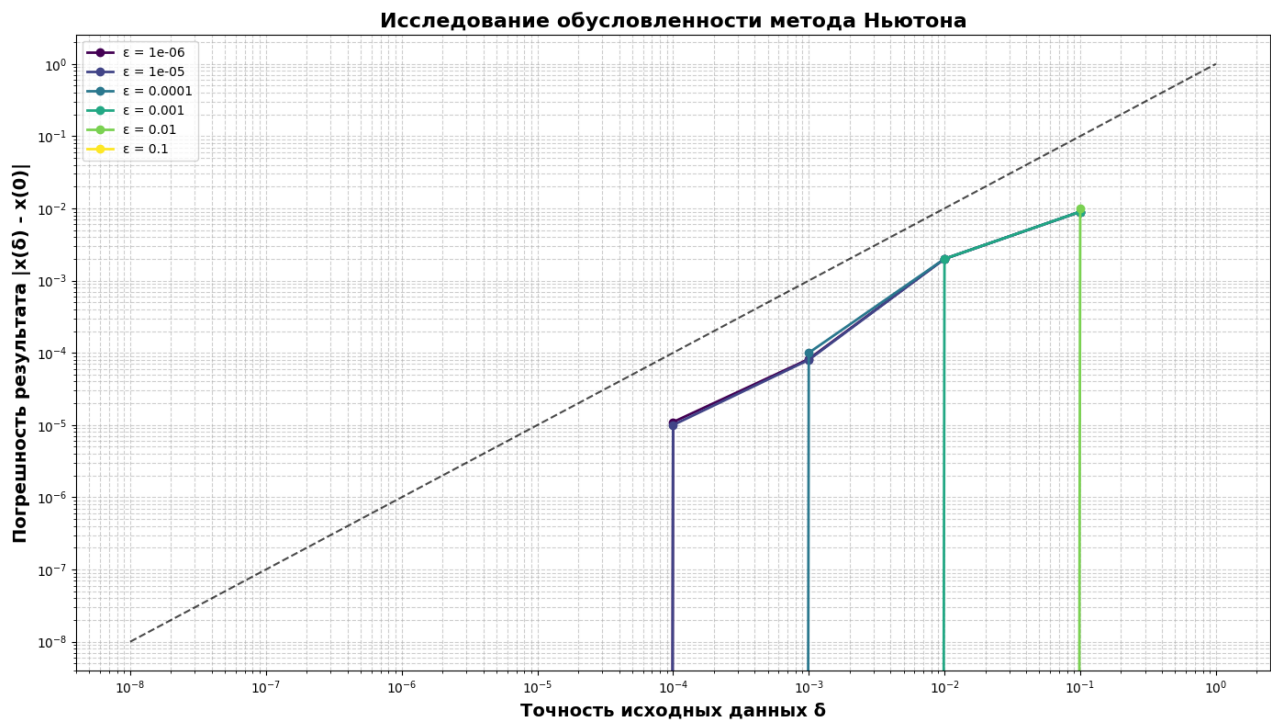


Рис. 5 - Зависимость результата вычислений от точности исходных данных

Погрешность результата при различной точности вычислений примерно соответствуют линейной зависимости, что говорит о хорошей обусловленности задачи.

Выводы.

В ходе выполнения лабораторной работы изучены методы бисекции, хорд, простых итераций и метод Ньютона, написана программа с использованием данных методов вычисляющая результат функции с различной точностью. Проведены исследование скорости сходимости и также обусловленности метода.

ПРИЛОЖЕНИЕ А.

ИСХОДНЫЙ КОДЫ ПРОГРАММ

```
#include <iostream>
#include <fstream>
#include <iomanip>
#include <cmath>

double comp_func(double x){
    return sinh(x)-x+1;
}

double Bisect(double left, double right, double eps, int &n){
    double e=std::abs(eps)*2.0;
    double f_left=comp_func(left), f_right=comp_func(right);
    double x=(left+right)/2.0, y;

    if (f_left*f_right > 0.0){
        std::cout << "Неверное задание интервала\n";
        exit(1);
    }

    if (eps <= 0.0){
        std::cout << "Неверное задание точности\n";
        exit(1);
    }

    n = 0;

    if (f_left == 0.0)
        return left;
    if (f_right == 0.0)
        return right;

    while ((right-left) >= e){
        x = 0.5*(right+left);
        y = comp_func(x);

        if (y == 0.0)
            return (x);

        if (y*f_left < 0.0)
            right = x;
        else{
            left = x;
            f_left = y;
        }
        n++;
    }
    return x;
}

double Round(double x, double delta){
    if (delta <= 1e-9){
        std::cout << "Неверное задание точности округления\n";
        exit(1);
    }
}
```

```

    }

    if (x > 0.0)
        return (delta*(long((x/delta)+0.5)));
    else
        return (delta*(long((x/delta)-0.5)));
}

int main(){
    std::ofstream out("output.txt");

    int fix=1;
    double l=-2.0, r=-1.0;
    for (double eps=0.1; eps >= 0.000001; eps *= 0.1){
        int cnt_it;
        double ans=Bisect(l, r, eps, cnt_it);
        out << std::fixed << std::setprecision(fix) << eps << ' '
<< 0.0 << ' ' << cnt_it << ' ' << ans << '\n';

        for (double delta=0.1; delta > 1e-9; delta *= 0.1){
            ans = Bisect(Round(l, delta), Round(r, delta), eps,
cnt_it);
            out << std::fixed << std::setprecision(fix) << eps << '
' << delta << ' ' << cnt_it << ' ' << std::fixed <<
std::setprecision(fix) << ans << '\n';
        }
        fix++;
    }

    out.close();
}

#include <iostream>
#include <fstream>
#include <iomanip>
#include <cmath>

double comp_func(double x){
    return 1.5*x-0.5*sinh(x)-0.5;
}

double Round(double x, double delta){
    if (delta <= 1e-9){
        std::cout << "Неверное задание точности округления\n";
        exit(1);
    }

    if (x > 0.0)
        return (delta*(long((x/delta)+0.5)));
    else
        return (delta*(long((x/delta)-0.5)));
}

double Iter(double x0, double eps, int& n, double delta){
    if (eps <= 0.0){
        std::cout << "Неверное задание точности\n";
        exit(1);
    }

```

```

    }

    double x1=Round(comp_func(x0), delta), x2=Round(comp_func(x1),
delta);
    n = 2;
    while((x1-x2)*(x1-x2) > fabs((2*x1-x0-x2)*eps)){
        x0 = x1;
        x1 = x2;
        x2 = Round(comp_func(x1), delta);

        n++;
    }

    return x2;
}

int main(){
    std::ofstream out("output.txt");

    int fix=1;
    double x=-1.5;
    for (double eps=0.1; eps >= 0.000001; eps *= 0.1){
        int cnt_it;
        double ans=Iter(x, eps, cnt_it, 1e-8);
        out << std::fixed << std::setprecision(fix) << eps << ' '
<< 1e-8 << ' ' << cnt_it << ' ' << ans << '\n';
        std::cout << std::fixed << std::setprecision(fix) << eps <<
' ' << ans << '\n';

        for (double delta=0.1; delta > 1e-9; delta *= 0.1){
            ans = Iter(x, eps, cnt_it, delta);
            out << std::fixed << std::setprecision(fix) << eps << '
' << delta << ' ' << cnt_it << ' ' << std::fixed <<
std::setprecision(fix) << ans << '\n';
        }
        fix++;
    }

    out.close();
}

#include <iostream>
#include <fstream>
#include <iomanip>
#include <cmath>

double comp_func(double x){
    return sinh(x)-x+1;
}

double Horda(double left, double right, double eps, int& n){
    double f_left=comp_func(left), f_right=comp_func(right);
    double x, y;

    if (f_left*f_right > 0.0){
        std::cout << "Неверное задание интервала\n";
        exit(1);
    }
}

```

```

    }
    if (eps <= 0.0){
        std::cout << "Неверное задание точности\n";
        exit(1);
    }

    n=0;
    if (f_left == 0.0)
        return left;
    if (f_right == 0.0)
        return right;

    do{
        x = left-(right-left)*f_left/(f_right-f_left);
        y = comp_func(x);

        if (y == 0.0)
            return x;

        if (y*f_left < 0.0){
            right = x;
            f_right = y;
        }
        else{
            left = x;
            f_left = y;
        }

        n++;
    } while (fabs(y) >= eps);

    return x;
}

double Round(double x, double delta){
    if (delta <= 1e-9){
        std::cout << "Неверное задание точности округления\n";
        exit(1);
    }

    if (x > 0.0)
        return (delta*(long((x/delta)+0.5)));
    else
        return (delta*(long((x/delta)-0.5)));
}

int main(){
    std::ofstream out("output.txt");

    int fix=1;
    double l=-2.0, r=-1.0;
    for (double eps=0.1; eps >= 0.000001; eps *= 0.1){
        int cnt_it;
        double ans=Horda(l, r, eps, cnt_it);
        out << std::fixed << std::setprecision(fix) << eps << ' '
<< 0.0 << ' ' << cnt_it << ' ' << ans << '\n';

        for (double delta=0.1; delta > 1e-9; delta *= 0.1){

```



```

        ans = Horda(Round(l, delta), Round(r, delta), eps,
cnt_it);
        out << std::fixed << std::setprecision(fix) << eps << '
' << delta << ' ' << cnt_it << ' ' << std::fixed <<
std::setprecision(fix) << ans << '\n';
    }
    fix++;
}

out.close();
}

```

```

#include <iostream>
#include <fstream>
#include <iomanip>
#include <cmath>

double comp_func(double x){
    return sinh(x)-x+1;
}

double comp_deriv(double x){
    return cosh(x)-1;
}

double Round(double x, double delta){
    if (delta <= 1e-9){
        std::cout << "Неверное задание точности округления\n";
        exit(1);
    }

    if (x > 0.0)
        return (delta*(long((x/delta)+0.5)));
    else
        return (delta*(long((x/delta)-0.5)));
}

double Newton(double x, double eps, int& n, double delta){
    double y, y1, dx;
    n = 0;
    do{
        y = Round(comp_func(x), delta);
        if (y == 0.0)
            return (x);

        y1 = Round(comp_deriv(x), delta);
        if (y1 == 0.0){
            std::cout << "Производная обратилась в ноль\n";
            exit(1);
        }

        dx = y/y1;
        x -= dx;
        n++;
    }
    while (fabs(dx) > eps);
}

```

```

        return x;
    }

    int main(){
        std::ofstream out("output.txt");

        int fix=1;
        double x=-2;
        for (double eps=0.1; eps >= 0.000001; eps *= 0.1){
            int cnt_it;
            double ans=Newton(x, eps, cnt_it, 1e-8);
            out << std::fixed << std::setprecision(fix) << eps << ' '
<< 1e-8 << ' ' << cnt_it << ' ' << ans << '\n';
            std::cout << std::fixed << std::setprecision(fix) << eps <<
' ' << ans << '\n';

            for (double delta=0.1; delta > 1e-9; delta *= 0.1){
                ans = Newton(x, eps, cnt_it, delta);
                out << std::fixed << std::setprecision(fix) << eps << '
' << delta << ' ' << cnt_it << ' ' << std::fixed <<
std::setprecision(fix) << ans << '\n';
            }
            fix++;
        }

        out.close();
    }

```